Task1

```
#
class Sample:
    def __init__(self):
        self.var1 = "AI"
        self.var2 = 2026

obj = Sample()
print(f"Instance Namespace: {obj.__dict__}")

def student_data(student_id, name=None, student_class=None):
    print(f"Student ID: {student_id}")
    if name: print(f"Name: {name}")
    if student_class: print(f"Class: {student_class}")


class Student:
    pass

print(f"Type of Student: {type(Student)}")
print(f"Keys of __dict__: {Student.__dict__.keys()}")
print(f"Module Name: {Student.__module__}")

class Student: pass
class Marks: pass

s_obj = Student()
m_obj = Marks()

print(f"Is s_obj a Student? {isinstance(s_obj, Student)}")
print(f"Is Marks derived from object? {issubclass(Marks, object)}")

class Student:
    def __init__(self, name, marks):
        self.student_name = name
        self.marks = marks

    def display(self):
        for attr, value in self.__dict__.items():
            print(f"{attr}: {value}")

s1 = Student("Zain", 85)

setattr(s1, 'student_class', 'AI-Batch-1')

delattr(s1, 'student_name')


s2 = Student("Ayesha", 92)
s2.student_id = "CUI-001"
print("\n--- Student 2 Details ---")
s2.display()
import sys

class NormalStudent:
    def __init__(self, name, id):
        self.name = name
        self.id = id

class SlimStudent:
    __slots__ = ['name', 'id']
    def __init__(self, name, id):
        self.name = name
        self.id = id

s_normal = NormalStudent("Test", 1)
s_slim = SlimStudent("Test", 1)

print(f"Normal Object Size: {sys.getsizeof(s_normal.__dict__)} bytes") #
```

```
Instance Namespace: {'var1': 'AI', 'var2': 2026}
Type of Student: <class 'type'>
Keys of __dict__: dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
Module Name: __main__
Is s_obj a Student? True
Is Marks derived from object? True

--- Student 2 Details ---
student_name: Ayesha
marks: 92
student_id: CUI-001
Normal Object Size: 296 bytes
```

TASK2

```
class Trapezoid:
    def __init__(self, name, a, b, h):
        self.name = name
        self.a = a
        self.b = b
        self.h = h

    def calculate_area(self):
        return ((self.a + self.b) / 2) * self.h

class Parallelogram:
    def __init__(self, name, base, height):
        self.name = name
        self.base = base
        self.height = height

    def calculate_area(self):
        return self.base * self.height


shapes_data = [
    Trapezoid("Trap_1", 10, 20, 5),
    Trapezoid("Trap_2", 5, 15, 10),
    Parallelogram("Para_1", 10, 15),
    Parallelogram("Para_2", 20, 8)
]

areas_dict = {}

for shape in shapes_data:
    areas_dict[shape.name] = shape.calculate_area()


max_shape = max(areas_dict, key=areas_dict.get)

print("\n--- Area Analysis ---")
for name, area in areas_dict.items():
    print(f"{name}: {area}")

print(f"\nWinner: {max_shape} with Maximum Area of {areas_dict[max_shape]}")
```

```
--- Area Analysis ---
Trap_1: 75.0
Trap_2: 100.0
Para_1: 150
Para_2: 160

Winner: Para_2 with Maximum Area of 160
```